

Capstone Project: Fake News Detection using NLP

Deliverable: Data Report

Group: 2B

Contributors:

Daniel Mwaka

Abdirahman Yussuf

Eliana Kariuki

Gerald Wanjala

Marion Osong

Abstract

This project develops an automated system for classifying news articles as *fake* or *real* using state-of-the-art Natural Language Processing (NLP) techniques. Four labeled datasets (gossipcop_fake.csv, gossipcop_real.csv, politifact_fake.csv, politifact_real.csv) sourced from the **FakeNewsNet** repository were scrapped using BeautifulSoup and Requests. The scrapped text data, accompanying news_url, respective title, and news_type from each of the four datasets were concatenated to yield a working dataset (data.csv) with 16,044 rows and 4 columns. The working dataset was cleaned (pandas), preprocessed (spaCy), and prepared for modelling (SMOTE, Random Undersampling). Four models were built and evaluated. The text feature was vectorized using Word2Vec to train the baseline Logistic Regression, XGBoost, and LSTM neural network models. The text feature was vectorized using the RobertaTokenizer to finetune the pretrained transformer. The fine-tuned RoBERTa-base transformer was the best-fit, and most generalizable model for deployment based on comparative performance in macro-averaged recall, AUC, and accuracy on the test set. The Lime library was leveraged to explain the model's predictions at the feature level for randomly selected samples from the working dataset. The fine-tuned transformer was deployed using FastAPI, Streamlit, and Docker to avail a practical, and scalable solution for integration into content moderation pipelines to curb the spread of misinformation across news-hosting websites, and social media platforms.

1. Business Understanding

1.1 Overview

Misinformation, particularly in the form of fake news, spreads quickly across social media and online platforms, influencing public opinion, political stability, and individual decision-making. Manual verification is time-consuming and often impractical given the high volume of content produced daily. Consequently, the demand for automated fake news detection systems has surged.

1.2 Problem Statement

Online news dissemination platforms and social media networks lack scalable, real-time mechanisms to detect and flag fake news articles. Without intervention, this misinformation erodes trust, distorts facts, and can even incite social unrest. The challenge lies in creating a robust NLP-based solution that can generalize across domains and reliably distinguish between fake and real content.

1.3 Proposed Solution and Target Audience

This project proposes a **binary classification NLP pipeline** that processes raw news articles, extracts semantic features, and predicts whether a given article is fake or real. The target audience includes:

- **News platforms** (e.g., BBC, CNN, Fox News)
- **Social media platforms** (e.g., Facebook, X/Twitter)
- **Fact-checking organizations** (e.g., PolitiFact, Snopes)
- **News aggregators** (e.g., Google News, Apple News)

1.4 Project Objectives

1. **Acquire** and integrate reputable labeled fake/real news datasets.
2. **Preprocess** data and perform EDA.
3. **Develop** and train NLP models, including transformer-based architectures.
4. **Evaluate** models to determine best-fit alternative.
5. **Deploy** the best-fit, robust, and most generalizable model.

1.5 Success Criteria

- Achieve over 0.85 in **Recall (Fake)**, Macro-Averaged recall, and **Precision (Real)** above 0.85.
- Ensure reproducibility and transparency in the modeling process (Lime Explanations).
- Demonstrate scalability by testing the deployed model on news articles published in 2025.

2. Data Understanding

The dataset originates from the **FakeNewsNet** repository, containing four labeled subsets:

- gossipcop_fake.csv – Fake celebrity news articles.
- gossipcop_real.csv – Verified celebrity news.
- politifact_fake.csv – Fake political news.
- politifact_real.csv – Verified political news.

Features:

- **id** – Unique article identifier.
- **news_url** – Article URL.
- **title** – Headline text.
- **tweet_ids** – IDs of tweets referencing the article.
- **publish_date** – Date of publication.
- **content** – Full news text.
- **type** – Label indicating *fake* or *real*.

Exploration Insights:

- Substantial imbalance between news articles that were successfully scrapped.
 - Gossip = 15504
 - Political = 540
- *GossipCop* articles are more verbose and entertainment-focused, whereas *PolitiFact* articles are politically centered.
- The vocabulary size is large, with considerable overlap in common function words but domain-specific differences in content words.
- Initial keyword frequency plots suggest that fake articles often contain sensational or exaggerated terms.

3. Data Preparation

Data preprocessing was crucial for improving model performance and interpretability. The following steps were undertaken:

1. **Merging Datasets** – Combined the four subsets into a single DataFrame with unified labeling (0 for fake, 1 for real).
2. **Missing Value Handling** – Dropped records with missing content or title fields, as these are critical for prediction.
3. **Text Cleaning:**
 - Removed HTML tags, URLs, and non-alphabetic characters.

- Converted all text to lowercase.
 - Expanded contractions (e.g., "can't" → "cannot").
4. **Tokenization & Lemmatization** – Used SpaCy for tokenizing words and reducing them to their base forms while preserving meaning.
 5. **Stopword Removal** – Removed high-frequency stopwords (e.g., “the”, “is”) to focus on semantically meaningful terms.
 6. **Vectorization** – Implemented **TF-IDF** for baseline models and **transformer tokenization** for advanced models.
 7. **Addressing Class Imbalance** – Applied **SMOTE** (Synthetic Minority Oversampling Technique) and **Random UnderSampling** for balancing during model training.
 8. **Train-Test Split** – Used an 80/20 split ensuring stratified sampling to preserve class proportions.

4. Modelling

Pipelines were leveraged to train each of the four implemented models with unbalanced data, SMOTE-balanced data, and RUS-balanced data. The SMOTE strategy was used to oversample the minority class while the RUS framework was adopted to randomly undersample the majority class.

Model	Recall (Fake)	Macro-averaged Recall	Precision (Real)
LR-RUS	0.8214	0.8614	0.8458
XGBoost-RUS	0.8427	0.8741	0.8622
LSTM-Imbalanced	0.7130	0.8277	0.7811
RoBERTa-RUS	0.8712	0.8816	0.8827

5. Evaluation

The Finetuned RoBERTa model using SMOTE-balanced dataset was ruled out as the best performing alternative because:

- It achieves the highest Recall (Fake) : Highest true positives for fake (class 0) predictions (2814).

- It achieves the highest Precision (Real) : Highest certainty (88.27%) for class 1 predictions when an article is actually real.

6. Deployment

Deployment considerations included:

- **API Endpoint** – The trained transformer model was wrapped into a FastAPI service to serve as the backend.
- **Streamlit Web App** – A user interface was created to prompt the entry of a news article's URL to check authenticity.
- **Maintenance Plan:**
 - Periodic retraining with newly collected labeled data.
 - Monitoring model drift to ensure consistent performance.
 - Updating preprocessing pipelines as online language evolves.

The deployed model is suitable for embedding into:

- Content moderation workflows for social media.
- Browser plugins to flag suspicious articles.
- Internal tools for editorial teams.